Heat equation solver

Description from KTH PDC Summer School 2014

Heat equation is a partial differential equation that describes the variation of temperature in a given region over time

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u$$

where u(x, y, z, t) represents temperature variation over space at a given time, and α is a thermal diffusivity constant.

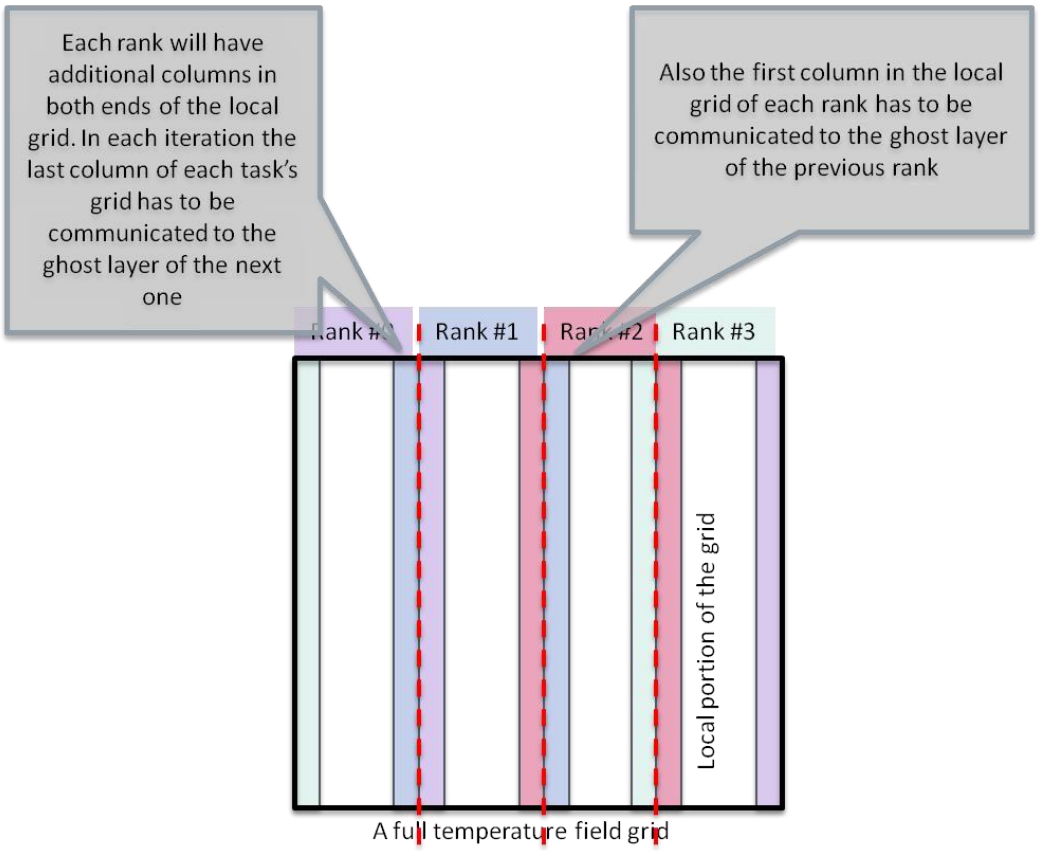
We limit ourselves to two dimensions (plane) and discretize the equation onto a grid. Then the Laplacian can be expressed as finite differences as

$$\nabla^2 u(i,j) = \frac{u(i-1,j) - 2u(i,j) + u(i+1,j)}{(\Delta x)^2} + \frac{u(i,j-1) - 2u(i,j) + u(i,j+1)}{(\Delta y)^2}$$

Where Δx and Δy are the grid spacing of the temperature grid u(i,j). We can study the development of the temperature grid with explicit time evolution over time steps Δt:

$$u^{m+1}(i,j) = u^m(i,j) + \Delta t \; \alpha \; \nabla^2 u^m(i,j)$$

Parallelization of the program is pretty straightforward, since each point in the grid can be updated independently. In distributed memory machines, the boundaries of the domains need to be communicated between tasks before each step:



There are solvers for the 2D equation implemented with C and Fortran in labs_c.tar.gz and labs_f.tar.gz. The solver develops the equation in a user-provided grid size and over the number of time steps provided by the user. The default geometry is a flat rectangle, but user can give other shapes as input files - a bottle is provided as an example (so you'll know how your beer will heat up when you take it to a sauna!).

You can compile the program by adjusting the Makefile as needed and typing "make".

Examples on how to run the binary:

- ./heat (no arguments - will run the program with default arguments, 256x256 grid and 500 time steps)
- ./heat bottle.dat (one argument - will run the program starting from a temperature grid provided in the file bottle.dat for the default number of time steps, i.e. 500)
- ./heat bottle.dat 1000 (two arguments - will run the program starting from a temperature grid provided in the file bottle.dat for 1000 time steps)
- ./heat 1024 2048 1000 (three arguments - will run the program in a 1024x2048 rectangular grid for 1000 time steps)

Remember that when working on Milner you can either run it interactively after booking a node with salloc, or with a job script.

The program will produce a .png image of the temperature field after every 100 iterations. You can change that from the parameter image_interval defined in heat_main.f90/c.